

Polyglot / Steganaliz Servisi — 4. Hafta Raporu (Gün 16-20)

Kapsam: Web Dashboard (Frontend) Entegrasyonu, Test & Raporlama

1. Yönetici Özeti

4. Hafta kapsamında, 3. Hafta'da tamamlanan FastAPI REST servisi bir **web dashboard**'a kavuştu ve proje kapanışa taşındı. Bağımsız, framework'süz bir sürükle-bırak arayüzü (`frontend/`) sıfırdan inşa edilip (Gün 16), gerçek backend'e `fetch` ile bağlanıp CORS yapılandırılarak tehdit skoru/özet/video player gösterimi tamamlandı (Gün 17). Sistem, gerçek HTTP API + web arayüzü katmanı üzerinden 12 dosyalık bir test matrisiyle (6 temiz + 6 polyglot) uçtan uca doğrulandı — %0 yanlış pozitif/negatif (Gün 18). Proje, README/docstring/ `docs/` indeksinin tamamlanmasıyla paylaşılabılır hale getirildi (Gün 19) ve son olarak Gün 1-19'un tamamının bağımsız bir kez daha doğrulandığı bir demo provası + staj final raporuyla (`docs/staj-raporu.md`) kapatıldı (Gün 20).

Beş günün (Gün 16-20) tamamı planlandığı gibi tamamlandı. Bu hafta boyunca `claude-in-chrome` tarayıcı otomasyonu bağlı olmadığından, frontend'in sürükle-bırak/tıklama etkileşimi görsel olarak otomatik doğrulanamadı; bunun yerine her gün frontend'in attığı isteğin birebiri HTTP düzeyinde (aynı `Origin`, aynı uç noktalar) test edilerek işlevsel olarak eşdeğer bir doğrulama yapıldı — bu sınırlama ve kullanıcının kendi tarayıcısında son bir görsel teyit yapması önerisi Gün 16'dan Gün 20'ye tutarlı şekilde not edildi.

2. Hafta 4 Hedefi

Proje planına göre bu haftanın çıktısı şu şekilde tanımlanmıştır:

"Tamamlanmış web tabanlı tespit paneli ve staj final raporu."

Bu hedef, 3. Hafta'nın REST API'sinin önce bağımsız bir arayüzle (Gün 16), sonra bu arayüzün API'ye canlı bağlanmasıyla (Gün 17), ardından bütün sistemin gerçek koşullarda doğrulanmasıyla (Gün 18) ve son olarak dokümantasyon (Gün 19) + kapanış/sunum (Gün 20) adımlarıyla tamamlandı.

3. Kullanılan Teknolojiler

Katman	Araç/Kütüphane
Frontend iskeleti	HTML5, CSS3 (framework'süz, koyu tema) (Gün 16)
Etkileşim	Vanilla JS (drag & drop, <code>fetch</code> , <code>AbortController</code>) (Gün 16-17)
Cross-origin istekler	FastAPI <code>CORSMiddleware</code> (Gün 17)
Medya oynatma	HTML5 <code><video></code> (Gün 17)
Doğrulama yöntemi	<code>curl</code> /HTTP düzeyinde CORS+şema testleri (<code>claude-in-chrome</code> yokluğunda) (Gün 17-18)
Dokümantasyon	Markdown raporlar, <code>docs/README.md</code> indeksi (Gün 19-20)

4. Gün 16 — Sürükle-Bırak Yükleme Arayüzü

Hedef: Backend'den bağımsız, sürükle-bırak + tıkla-seç ile dosya seçilebilen bir web arayüzü (`frontend/index.html`, `style.css`, `app.js`) oluşturmak.

`dropzone` elemanına `dragover` / `dragleave` / `drop` / `click` / `keydown` event'leri bağlandı; hem sürükle-bırak hem klasik dosya seçici aynı `handleFile()` fonksiyonundan geçiyor. İstemci tarafında backend'dekiyle birebir aynı doğrulama (yalnızca PNG/JPEG, 25 MB üst sınır, boş dosya reddi) UX amaçlı tekrarlandı — gerçek güvenlik sınırı backend'de kalmaya devam ediyor.

`URL.createObjectURL()` ile dosya önizleme kartı ve `setLoading()` ile (Gün 17'de kullanılacak) bir loading göstergesi hazırlandı.

Kontrol	Sonuç
<code>index.html</code> / <code>style.css</code> / <code>app.js</code> statik sunumda 200 OK	✓
DOM id'leri (<code>dropzone</code> , <code>file-input</code> , <code>error-message</code> , ...) <code>app.js</code> ile birebir eşleşiyor	✓ (elle kod incelemesi)

5. Gün 17 — Frontend-Backend Entegrasyonu ve Sonuç Gösterimi

Hedef: Arayüzü gerçek `/api/v1/analyze` endpoint'ine bağlamak; tehdit skorunu görsel bir göstergeyle, özeti metinle, gizli videoyu `<video>` player'ıyla göstermek; CORS'u yapılandırmak.

`backend/app/main.py` 'ye `CORSMiddleware(allow_origins=["*"])` eklendi (proje kimlik doğrulama/cookie kullanmadığı için development ortamında kabul edilebilir). `app.js` 'e `analyzeFile()` eklendi: dosya seçilir seçilmez otomatik `POST /api/v1/analyze`, `AbortController` ile yarım kalan isteklerin iptali, `try/finally` ile loading göstergesinin her

koşulda kapanması. `renderResults()` : durum rozeti (kırmızı/yeşil), `threat_score` bar'ı (yeşil/sarı/kırmızı eşikli), `analysis_summary` metni, `extracted_video_url` doluysa `<video controls>`.

Senaryo	Sonuç
<code>OPTIONS /api/v1/analyze</code> (CORS preflight), <code>Origin: 127.0.0.1:5500</code>	200, <code>access-control-allow-origin: *</code>
<code>POST</code> polyglot PNG, aynı <code>Origin</code>	201, <code>threat_score=76</code> , video URL dolu
Ayıklanan video URL, aynı <code>Origin</code> ile <code>GET</code>	200, <code>content-type: video/mp4</code>
Temiz JPEG	<code>polyglot_status=false</code> , <code>threat_score=2</code> , video URL <code>null</code>

6. Gün 18 — Uçtan Uca Doğrulama Testleri

Hedef: Sistemi CLI değil, gerçek dağıtılmış web uygulaması (backend + frontend + CORS) üzerinden en az 5 temiz + 5 polyglot dosyayla doğrulamak.

Gün 10'un 10 dosyalık matrisi yeniden kullanıldı; genuine yeni kapsam için `make_polyglot.py` ile iki yeni taşıyıcıdan (checkerboard PNG, düz renk JPEG) iki yeni polyglot üretildi — toplam **12 dosya (6 temiz + 6 polyglot)**. Her dosya için `frontend/app.js` 'in attığı isteğin birebiri (`multipart/form-data`, `Origin` header'ı dahil) gönderildi; ayıklanan video URL'leri de aynı `Origin` ile `GET` edilip oynatılabilirliği doğrulandı.

Sonuç	Değer
Doğru <code>polyglot_status</code>	12/12
Yanlış pozitif (temiz dosyalarda)	0/6 (%0)
Yanlış negatif (polyglot dosyalarda)	0/6 (%0)
<code>500</code> hatası (14 istek boyunca)	0

Bu turda kod değişikliği gerektiren hiçbir hata/eksik bulunmadı.

7. Gün 19 — Dokümantasyon ve GitHub Deposu Düzenleme

Hedef: Projeyi paylaşılabılır hale getirmek: `README.md` 'nin tamamlanması, çekirdek fonksiyonlara docstring eklenmesi, `docs/` klasörünün gezinilebilir bir indeksle düzenlenmesi.

README.md 'ye "Mimari / İstek Akışı" alt bölümü eklendi (istek katmanlarının tam izini gösteren diyagram). Python `ast` ile taranıp `scripts/*.py` ve `backend/app/*.py` 'deki 12 dosyada ~29 çekirdek fonksiyona (CLI iskeleti hariç) docstring eklendi. `docs/README.md` indeksi oluşturuldu: referans dokümanlar, günlük rapor tablosu, haftalık özetler, üretilmiş görsellerin kaynağı — hiçbir dosya taşınmadı (raporlardaki görelî görsel linkleri kırılmasın diye).

Kontrol	Sonuç
Docstring eklemeleri sözdizimi (<code>py_compile</code>)	✅ Hatasız
README'deki kurulum+API adımları canlı sunucuya karşı	✅ (<code>threat_score=76</code> , örnekle birebir aynı)

8. Gün 20 — Sunum, Demo Provası ve Staj Raporu Teslimi

Hedef: Kısa bir demo akışı hazırlayıp prova etmek; staj final raporunu (`docs/staj-raporu.md`) tamamlamak.

Bu raporun da hazırlandığı oturumda, Gün 1-19'un **tamamı** bağımsız olarak yeniden test edildi: ortam kontrolü, CLI pipeline (`analyze.py` , `extract.py` , `video_metadata.py`), ve gerçek bir `uvicorn` sunucusuna karşı tüm API senaryoları (health-check, temiz/polyglot analiz, hata kodları, `/media` video servisi). Tüm sonuçlar (`threat_score` : 0/76/85, offset'ler, entropy delta'ları) önceki günlerin raporlarıyla birebir eşleşti. Ardından `docs/staj-raporu.md` (özet, yöntem, karşılaşılan zorluklar, sonuçlar) ve `docs/gun20-demo-ve-kapanis-raporu.md` yazıldı; README.md / `docs/README.md` / `PLAN.md` "Gün 20/20 tamamlandı" olarak güncellendi.

Kontrol	Sonuç
Gün 1-19 bağımsız yeniden doğrulama	✅ Hiçbir hata/tutarsızlık bulunmadı
Demo akışı (temiz → polyglot → analiz → ayıklama → oynatma)	✅ HTTP düzeyinde baştan sona hatasız
<code>docs/staj-raporu.md</code>	✅ Tüm bölümleriyle tamamlandı

9. Hafta 4 Doğrulama Özeti

Gün	Doğrulama Yöntemi	Sonuç
16	Statik dosya sunumu (<code>curl</code>) + elle DOM/id eşleşme incelemesi	Geçti
17	CORS preflight + gerçek <code>POST</code> , <code>Origin</code> header'ıyla	Geçti — şema ve video erişimi doğru

Gün	Doğrulama Yöntemi	Sonuç
18	12 dosyalık test matrisi, gerçek HTTP API + CORS	Geçti — 12/12, %0 FP/FN
19	<code>py_compile</code> + canlı sunucuya karşı README adımları	Geçti — örnekle birebir aynı çıktı
20	Gün 1-19'un tamamının yeniden çalıştırılması	Geçti — hiçbir sapma yok

Her günde tutarlı olarak not edilen tek sınırlama: bu ortamda canlı tarayıcı otomasyonu (`claude-in-chrome`) bulunmadığından, gerçek sürükle-bırak/tıklama etkileşimi görsel olarak otomatik doğrulanamadı. HTTP düzeyinde doğrulama (frontend'in attığı isteğin birebiri) işlevsel olarak eşdeğer kabul edildi; kullanıcının kendi tarayıcısında en az bir temiz ve bir polyglot dosyayla son bir görsel teyit yapması önerisi hafta boyunca korundu.

10. Karşılaşılan Zorluklar ve Öğrenilen Dersler

- **Tarayıcı otomasyonu olmadan frontend doğrulaması (Gün 16-20):** Görsel etkileşimi otomatik test edememek başlangıçta bir eksiklik gibi görünse de, frontend'in ürettiği isteğin HTTP düzeyinde birebir tekrarlanması (aynı `Origin`, aynı `multipart/form-data` şeması) entegrasyon katmanının (CORS, şema, video servisi) doğruluğunu kanıtlamak için yeterli oldu. Ders: görsel bir arayüzün *davranışını* doğrulamakla, o davranışın *sonucu* olan ağ isteğini doğrulamak çoğu zaman eşdeğer güvence sağlar; ikisi arasındaki fark yalnızca "kullanıcı deneyimi hissi"nde kalır ve bu, kullanıcıya bırakılan son bir manuel teyitle kapatılabilir.
- **`allow_origins=["*"]` kararı (Gün 17):** `frontend/` 'in build adımsız, `file://` veya herhangi bir statik port üzerinden açılabilmesi gerektiği için `origin`'i tek bir değere sabitlemek gereksiz kısıtlama olurdu; bu yalnızca geliştirme ortamı için kabul edilebilir bir tercih olarak işaretlendi (production'a taşınırsa gözden geçirilmeli).
- **Gün 18'in "yeniden test" ile "yeni kapsam" dengesi:** Gün 10'daki 10 dosyanın çoğunu yeniden kullanmak bilinçli bir tercihti (amaç algoritmayı değil entegrasyon katmanını test etmektir), ama sıfır yeni kapsam eklemek de yanıltıcı olurdu — bu yüzden 2 yeni taşıyıcıdan üretilen polyglot eklendi. Ders: regresyon testinde eski senaryoları tekrarlamak değerlidir, ama en az bir yeni girdiyle "sadece ezberlenmiş veriyle çalışıyor" ihtimalini de ekarte etmek gerekir.
- **Gün 20'de "iddia edilen" ile "gerçekten çalışan" arasındaki fark:** Plan kutucukları `[x]` olsa da, staj raporunu yazmadan önce Gün 1-19'un tamamının yeniden çalıştırılması bilinçli bir tercihti — statik bir inceleme (dosyaların var olduğunu görmek) ile kodun hâlâ iddia edilen sonucu ürettiğini görmek aynı şey değil. Bu tur hiçbir sapma bulmadı, ama bulunmuş olsaydı staj raporu yanlış bir "her şey tamam" iddiasıyla yazılmış olurdu.

11. Hafta 4 Çıktısı

Plan'da tanımlanan hedefe ulaşıldı: 3. Hafta'nın REST API'si, sürükle-bırak ile dosya yükleyip tehdit skorunu/özetini/gizli videoyu gösteren **tamamlanmış bir web tabanlı tespit paneli** hâline geldi; sistem 12 dosyalık gerçek bir test matrisiyle %0 FP/FN ile doğrulandı; proje dokümantasyonu (README, docstring'ler, docs/ indeksi) tamamlandı; ve **staj final raporu** (docs/staj-raporu.md) teslim edildi. 20 günlük plan bu haftayla birlikte baştan sona tamamlanmış ve bağımsız olarak doğrulanmıştır.

12. Ekler — Üretilen Dosyalar

- docs/gun16-surukle-birak-arayuz-raporu.md/pdf
- docs/gun17-frontend-backend-entegrasyonu-raporu.md/pdf
- docs/gun18-uctan-uca-dogrulama-raporu.md/pdf
- docs/gun19-dokumantasyon-repo-raporu.md/pdf
- docs/gun20-demo-ve-kapanis-raporu.md/pdf
- docs/staj-raporu.md — staj final raporu
- docs/test-sonuclari.md — Gün 18 bölümü eklendi
- frontend/index.html , frontend/style.css , frontend/app.js
- backend/app/main.py — CORSMiddleware eklendi
- docs/README.md — docs/ klasörü indeksi